Domain-Specific Fine-Tuning for Reliability Engineering: From Small-Seed Self-Instruct to Reinforcement Learning

Anonymous CVPR submission

Paper ID

Abstract

Large language models show strong general reasoning but struggle with domain-specific technical tasks such as reliability engineering, where answers require precise numerical computation and specialized knowledge. We present a complete pipeline for adapting a general-purpose LLM to the reliability engineering domain, starting from only 56 seed questions extracted from textbooks. We introduce an adapted self-instruct methodology that replaces same-model answer consistency with cross-model verification, combined with paraphrase-based augmentation inspired by MetaMath, to scale the dataset to 866 high-quality question-reasoning-answer triplets. We identify the self-instruct ceiling effect, wherein LLM generators tend to produce questions systematically easier than real domain problems. Supervised fine-tuning of Qwen3-8B with QLoRA yields a statistically significant $+18.6\%$ accuracy improvement ($p = 0.002$) under 10-fold cross-validation, placing our 8B-parameter model between frontier models o3-mini and Gemini 3.1 Pro on reliability engineering tasks. We further apply Group Relative Policy Optimization (GRPO) with verifiable rewards on 266 "mixed-difficulty" questions, achieving an additional $+7.5$ percentage points over the base model on in-distribution questions. Our ablation study reveals a paradoxical role for SFT: it degrades standalone performance on hard questions (-20.4pp on holdout) yet is essential as GRPO initialization ($+7.1$pp vs. GRPO from scratch). Crucially, GRPO preserves base-model generalization on a 54-question independent holdout (53.7%), fully recovering the severe catastrophic forgetting caused by SFT alone (-20.4pp), demonstrating that RL from verifiable rewards can improve domain performance without the out-of-distribution cost of supervised fine-tuning.

1. Introduction

Reliability engineering is concerned with the analysis and prediction of failure in complex systems—from nuclear power plants and aircraft to electrical grids and autonomous vehicles. Practitioners routinely employ sophisticated mathematical tools including Weibull and exponential lifetime distributions, Markov chain modeling, Bayesian inference, fault tree analysis, and system-level reliability computation for series-parallel and k-out-of-n architectures. Mastering these techniques requires years of specialized training, and even experienced engineers benefit from computational assistance when solving multi-step quantitative problems.

Large language models (LLMs) have demonstrated impressive general reasoning capabilities across a wide range of tasks [9]. However, when applied to domain-specific technical problems such as reliability engineering, their performance degrades substantially. Prior work evaluated state-of-the-art models on a set of hand-written reliability engineering problems and found that even frontier models fail to produce correct numerical answers in the majority of cases, often generating plausible-sounding but mathematically incorrect solutions. This performance gap motivates the development of domain-adapted models that can reliably assist with reliability engineering computations.

A central challenge in building such models is the scarcity of training data. Unlike well-studied domains such as general mathematics [5, 24] or medicine [17], reliability engineering lacks large-scale question-answer datasets or established benchmarks. Manually creating such datasets is prohibitively expensive, as each problem requires expert knowledge to both formulate and solve. This creates a bootstrapping problem: we need data to train a model, but generating quality data requires the very expertise the model is meant to provide.

We address this challenge with a data generation and fine-tuning pipeline that bootstraps 56 textbook-extracted seed questions into 866 verified training examples, achieving $+18.6\%$ accuracy ($p = 0.002$) on Qwen3-8B. Our con-

*Code and data: https://github.com/ADnocap/Reliability-Domain-Specific-LLM

†Experiments (exp) described in appendix A

tributions are:

- A cross-model answer verification method for synthetic data quality control, replacing same-model consistency checks that we show are trivially satisfied in technical domains (Section 3.2).
- The identification and empirical characterization of the self-instruct ceiling effect (Section 3.2), and evidence that paraphrase augmentation overcomes it more effectively than difficulty scaling (Section 3.4).
- A reproducible SFT pipeline achieving $+18.6\%$ on reliability engineering with a fine-tuned 8B model competitive with frontier models an order of magnitude larger (Section 4).
- Integration of GRPO with verifiable rewards and DAPO-style dynamic sampling [25] to improve domain reasoning by $+7.5$pp beyond the base model, with an ablation showing that SFT warm-start is essential for RL effectiveness despite degrading standalone performance (Section 5).

2. Related Work

2.1. Domain-Specific LLM Adaptation

Fine-tuning LLMs for specialized domains has shown promising results across several fields. In medicine, Med-PaLM [17] achieved expert-level performance on medical question answering through instruction tuning on curated clinical datasets, while PMC-LLaMA [21] demonstrated that continued pre-training on biomedical literature followed by instruction tuning can substantially improve domain performance. In the legal domain, models fine-tuned on case law and statutory text have shown improved performance on legal reasoning tasks [1]. For code generation, Code Llama [14] showed that domain-specific continued pre-training followed by instruction tuning outperforms general-purpose models.

A common finding across these efforts is that domain adaptation is most effective when high-quality, domain-specific training data is available. However, for niche technical fields like reliability engineering where experts are proficient in multiple technical disciplines such as probability theory, statistics, and systems engineering, no such datasets or benchmarks currently exist. Our work addresses this gap by developing both a data generation pipeline and a fine-tuning methodology specifically designed for low-resource technical domains.

2.2. Self-Instruct and Synthetic Data Generation

Self-Instruct [20] introduced a framework for bootstrapping instruction-following data by using a language model to generate its own training examples from a small seed set of 175 human-written tasks. The method produced 52K instruction-output pairs, improving GPT-3's instruction following by $+33.1\%$ on Super-NaturalInstructions. To ensure diversity, generated instructions are only added to the task pool if their ROUGE-L similarity with all existing instructions falls below 0.7. However, this diversity filter says little about *correctness*, in fact, only 54% of generated examples were rated as fully valid, with output quality identified as the weakest component.

Several works have extended this framework. Stanford Alpaca [18] scaled self-instruct to generate 52K examples using GPT-3.5 for fine-tuning LLaMA. WizardLM [22] introduced Evol-Instruct, which iteratively increases instruction complexity through depth and breadth evolution. Ranaldi and Freitas [13] proposed Self-Refine Instruction-Tuning, a two-stage approach where LLMs first generate chain-of-thought demonstrations for smaller student models, which then self-refine via DPO.

For verifiable tasks such as mathematics, answer-consistency filtering has been proposed as a quality control mechanism [23]. The idea is straightforward: after generating a question-answer pair, the same model is prompted to solve the question independently a n times; if more than $n/2$ instances agree on the anwer, the example is considered reliable and retained, otherwise it is discarded. This targets the correctness gap left by ROUGE-L, which only filters for diversity. However, a subtle issue arises when the same model is used for both generation and verification. We observe empirically that models can already solve the vast majority of questions they generate. In our domain, baseline models achieve 92–97% accuracy on self-instruct-generated questions while scoring only 68–76% on real textbook problems. This means same-model answer consistency is trivially satisfied and does not filter for difficulty or correctness in any meaningful way. While this does not invalidate the downstream fine-tuning results of these approaches (the trained model may still improve through exposure to diverse reasoning patterns), it reveals that answer consistency is not a reliable proxy for data quality in technical domains.

This observation is related to a broader limitation identified by SearchInstruct [8]: self-instruct "depends solely on internal knowledge," meaning the generator can only create questions within its own capability envelope. They propose grounding generation in retrieved documents to overcome this.

Our approach addresses the correctness problem through *cross-model* answer verification: an independent model (Claude Opus 4) generates questions from the seed dataset and solves them, we then use a comparably powerful model (ChatGPT 5.4) to verify the solution by solving each question without seegin the answer or reasoning trace, and only items where both models agree on the numerical answer (within 5% tolerance) are retained. Because the verifier is a different model with different failure modes, agree-

ment provides a stronger signal than same-model consistency. We additionally use a stronger generator model than the fine-tuning target, and generate full chain-of-thought triplets (question, reasoning, answer) rather than instruction-output pairs.

2.3. Data Augmentation for Multi-Step Mathematical Reasoning

Data augmentation has emerged as a critical strategy for improving mathematical reasoning in LLMs. MetaMath [24] demonstrated that rephrasing mathematical questions from multiple perspectives (without changing the underlying problem) significantly improves reasoning performance, achieving state-of-the-art results on GSM8K and MATH benchmarks. The key insight is that surface-level diversity in question formulation is more valuable than increasing problem difficulty. PersonaMath [19] extended this idea by generating question variants from different persona perspectives.

Ding *et al*. [4] provide a comprehensive survey of LLM-based data augmentation, finding that while larger generator models produce higher-quality data, cross-model validation is essential to prevent self-reinforcing biases. Shumailov *et al*. [16] formalized this concern as "model collapse," showing that training on model-generated data without external grounding leads to progressive quality degradation across generations.

Our paraphrase augmentation strategy is directly inspired by MetaMath: we use Claude Opus 4.6 to rephrase questions while strictly preserving all numerical values and mathematical relationships, then verify meaning preservation with GPT-5.4. This approach scales our dataset from 280 to 866 examples and yields our strongest results ($+18.6\%$, $p = 0.002$), consistent with the finding that surface diversity outperforms difficulty scaling.

2.4. Reinforcement Learning for Reasoning

Reinforcement learning from human feedback (RLHF) [10] established the paradigm of aligning LLMs with human preferences through reward modeling and policy optimization. Direct Preference Optimization (DPO) [12] simplified this by eliminating the need for a separate reward model, directly optimizing the policy from preference pairs. However, both methods rely on human-generated or model-generated preference data, which can be noisy for technical domains.

For tasks with verifiable answers, such as mathematics, reinforcement learning from verifiable rewards offers a compelling alternative. DeepSeek-Math [15] introduced Group Relative Policy Optimization (GRPO), which generates multiple candidate responses per question, scores them against ground-truth answers, and computes group-relative advantages without requiring a critic model. DeepSeek-R1 [2] scaled this approach, demonstrating that GRPO can elicit sophisticated reasoning behaviors including self-verification and chain-of-thought refinement. DAPO [25] further introduced *dynamic sampling*: when all G generations produce identical rewards (zero variance), the prompt is discarded and resampled, ensuring every training step provides a meaningful gradient. This is particularly important for small datasets where many questions are either too easy or too hard for the current policy.

Reliability engineering problems are particularly well-suited for GRPO because they produce verifiable numerical answers that can serve as ground-truth rewards. In our pipeline, SFT first provides domain knowledge and reasoning format, after which GRPO optimizes the model to prefer reasoning chains that lead to correct final answers. Our early experiments with GRPO on small datasets (215 questions) showed limited improvement over SFT alone ($+0.5\%$ vs. $+2.3\%$), motivating the use of larger augmented datasets for the RL stage.

Our GRPO experiments (Section 5) confirm that GRPO is effective for reliability engineering when paired with SFT initialization and a carefully screened training set targeting the model's learning frontier. We further show that SFT warm-start is essential: GRPO from a fresh LoRA produces negligible gains, while GRPO from SFT achieves $+7.5$pp on mixed-difficulty questions.

2.5. Parameter-Efficient Fine-Tuning

Low-Rank Adaptation (LoRA) [6] enables efficient fine-tuning by adding trainable low-rank matrices to frozen pre-trained weights, drastically reducing the number of trainable parameters. QLoRA [3] combines LoRA with 4-bit quantization, enabling fine-tuning of large models on consumer-grade hardware. NEFTune [7] demonstrated that adding uniform noise to embedding vectors during fine-tuning acts as an effective regularizer, improving instruction-following performance by 25–35% on standard benchmarks.

We employ 4-bit QLoRA with LoRA applied to all attention and MLP projection layers, combined with NEFTune ($\alpha = 5$) as a regularizer against overfitting on our small training sets (approximately 780 examples per fold in 10-fold cross-validation on 866 total examples).

3. Data Pipeline

Our data pipeline transforms 56 textbook-extracted seed questions into 866 verified training examples through four stages: seed extraction, synthetic generation with cross-model verification, cleaning, and paraphrase augmentation. Figure 1 illustrates the complete pipeline.

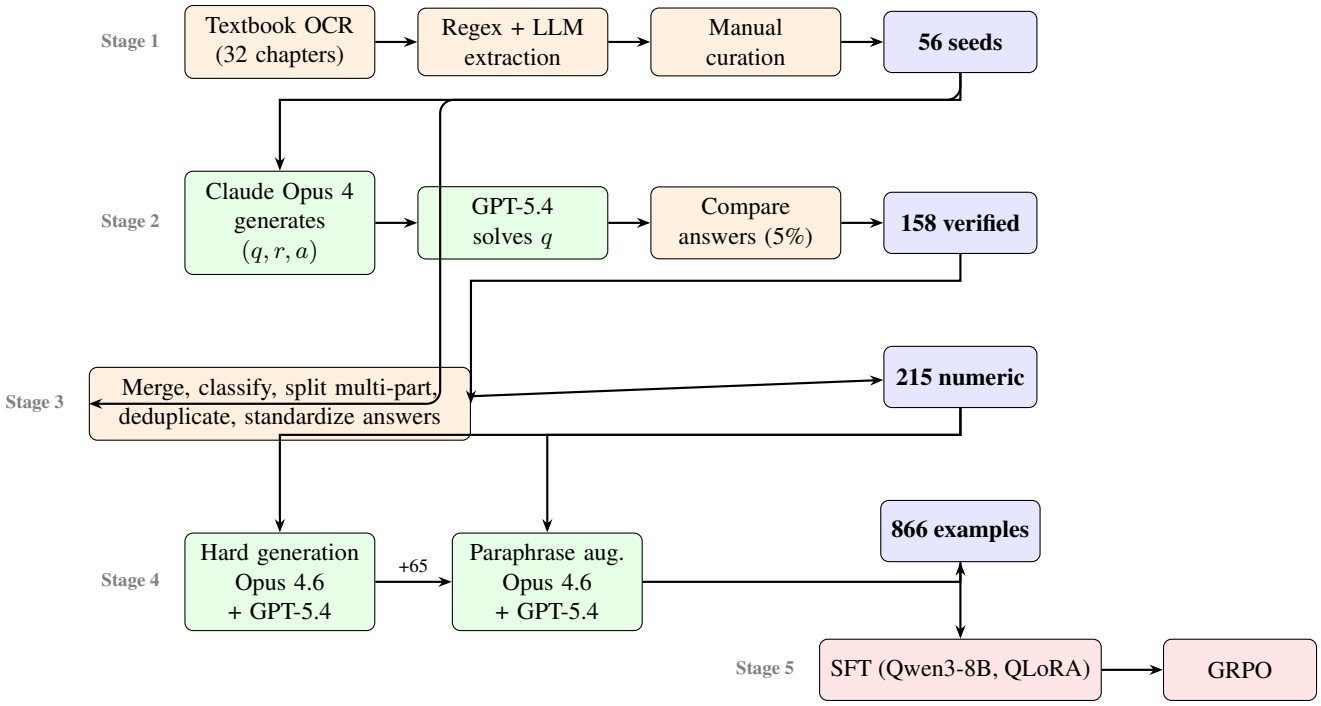

Figure 1. Data pipeline overview. Blue: dataset milestones; orange: processing; green: LLM-based generation/verification; red: training. Seed questions feed both the generation stage and the cleaning stage (routed via the left margin to avoid crossing Stage 2 nodes). **[TODO: FIX THE FUCKING ARROWS]**

3.1. Seed Dataset Extraction

We digitized 32 chapters from reliability engineering textbooks using Mistral OCR, converting them to structured markdown. The textbooks cover exponential and Weibull lifetime distributions, mean time to failure (MTTF) and hazard functions, Bayesian inference, series-parallel and k-out-of-n system reliability, reliability demonstration testing, and acceptance sampling.

[TODO: need to quote and mention which textbook, I cant remember which it is and maybe we worl this part above]

From this corpus, we extracted question-reasoning-answer (QRA) triplets through a three-step process: (1) keyword scanning with regular expressions to identify exercise and solution blocks, (2) LLM-assisted validation to check self-containment (no references to figures or prior parts), and (3) structured extraction into (q, r, a) triplets using GPT-5-Mini. After iterative cleaning—including LaTeX normalization, Unicode fixing, and manual curation—we obtained **56 high-quality seed QRA triplets** covering the core topics of reliability engineering.

3.2. Adapted Self-Instruct with Cross-Model Verification

3.2.1. Generation with a Stronger Model

Unlike the original Self-Instruct [20], which uses the same model for both generation and fine-tuning, we employ *stronger* generator models to create training data for a smaller target model (Qwen3-8B). This asymmetry is motivated by the observation that the target model lacks the domain knowledge needed to generate correct reliability engineering problems and solutions.

Each generation prompt includes 3 randomly sampled seed examples and requests 2 new problems. The generator is instructed to produce complete (q, r, a) triplets with step-by-step chain-of-thought reasoning, using temperature 0.8 for diversity. Quality gates filter out questions shorter than 30 characters, reasoning shorter than 50 characters, placeholder answers, and near-duplicates (by 200-character question prefix). This process yielded 158 synthetic QRA triplets that passed initial quality checks.

3.2.2. Cross-Model Answer Verification

To ensure mathematical correctness, we replace same-model answer consistency [23] with a cross-model verification pipeline:

1. **Generate and solve:** Claude Opus 4 produces a (q, r, a) triplet from the seed pool (temperature 0.8).

Table 1. Accuracy of baseline models on textbook-extracted seed questions (56) vs. self-instruct-generated questions, revealing the systematic difficulty gap.

Model	Textbook (56)	Synthetic
Qwen3-8B	68.2%	92.4%
GPT-4-Mini	76.0%	97.1%

[TODO: Verify exact numbers from evaluation logs. idf these numbers chnage, you need to chnage the referacnes to this below and in the intro]

2. **Verify:** GPT-5.4 independently solves the same question without seeing the generator's answer or reasoning (temperature 0.1).
3. **Compare:** Final numerical answers are compared with 5% relative tolerance, with LLM-based semantic comparison as fallback for edge cases.

Only items where both models agree (MATCH) are retained. A safety valve halts generation if the acceptance rate drops below 10% after 20 batches. Because the generator and verifier are different models with different architectures and failure modes, agreement provides a substantially stronger correctness signal than same-model consistency.

3.2.3. The Limitations of Same-Model Answer Consistency

A key motivation for cross-model verification is our empirical observation that same-model answer consistency, as used in prior work [23], is trivially satisfied for self-instruct-generated questions. Table 1 shows that baseline models achieve 92–97% accuracy on synthetic questions while scoring only 68–76% on the 56 textbook-extracted seed problems.

This means that if the generating model is also used for answer consistency verification, it will confirm nearly all of its own outputs, regardless of whether they represent genuinely challenging or educationally useful problems. The verification step thus filters for self-consistency rather than for quality or difficulty. This does not invalidate the downstream fine-tuning improvements reported in prior work as the trained model may still benefit from exposure to diverse reasoning patterns, but it reveals that answer consistency is not a reliable proxy for data quality in technical domains where the goal is to teach *new* knowledge rather than to align instruction-following behavior.

3.2.4. Self-Instruct Ceiling Effect

The accuracy gap in Table 1 reflects a more fundamental limitation that we term the *self-instruct ceiling effect*: LLM generators create questions within their own capability envelope. The generator avoids edge cases it does not understand (truncated distributions, competing risks), defaults to standard formulations matching its training distribution, and produces clean numerical answers that do not require iterative methods or lookup tables. Consequently, synthetic questions are systematically easier than real domain problems.

This has a direct implication for fine-tuning: training on self-instruct data may teach the model the *format* of domain-specific reasoning (step-by-step solutions, correct use of notation) without imparting the *knowledge* needed to solve genuinely hard problems. This ceiling effect motivates our paraphrase augmentation strategy (Section 3.4): rather than generating more synthetic questions, we create surface-level rephrases of real textbook problems, preserving their difficulty while increasing training set diversity.

3.3. Dataset Cleaning and Standardization

The combined seed and verified synthetic data (140 items) underwent LLM-assisted cleaning using Claude Opus 4.6 (temperature 0.1):

1. **Type classification:** Each item was labeled as SINGLE_NUMERIC, MULTI_PART_NUMERIC, FORMULA, DERIVATION, or CONCEPTUAL.
2. **Multi-part splitting:** 107 multi-part questions were split into fully self-contained entries, with each sub-question rewritten to include all necessary context (removing references such as "from Part (a)").
3. **Answer standardization:** Trailing periods, units, and commas were stripped; percentages retained as-is; LaTeX notation cleaned (e.g., `\frac{14}{33}` → `14/33`).
4. **Deduplication:** 173 near-duplicate entries removed by 200-character question prefix matching.

The resulting **master dataset** contains **256 items**: 215 numeric, 21 formula, 19 text, and 1 boolean. For all fine-tuning experiments, we use the 215 numeric questions as the base, since these permit automated evaluation via numerical comparison.

3.4. Paraphrase Augmentation

3.4.1. Augmentation Strategy

Inspired by MetaMath [24] and PersonaMath [19], we augment the dataset by generating surface-level rephrases of existing questions while strictly preserving mathematical content. Each original question is rephrased by Claude Opus 4.6 (temperature 0.7) with explicit constraints: all numerical values, formulas, and intermediate calculations must remain identical; only wording, sentence structure, and scenario framing may change (e.g., "plant" → "factory", "compute" → "determine"). Two manually curated few-shot examples guide the model.

Each rephrase is then verified by GPT-5.4 (temperature 0.1), which checks five dimensions: numerical values preserved, mathematical meaning preserved, constraints preserved, solution equivalence, and reasoning equivalence. A

Table 2. Dataset versions and their composition. All entries are numeric QRA triplets with verified answers.

Version	File	Size	Source
v1 (cleaned)	`cleaned.jsonl`	215	98 textbook + 158 verified
v2 (+ hard)	`v2.jsonl`	280	v1 + 65 hard generated
v3 (+ para.)	`v3.jsonl`	501	v2 + 221 paraphrased
v4 (+ para.)	**`v4.jsonl`**	**866**	**v2 + 586 paraphrased**

Table 3. Base model comparison on 215 numeric questions (5-fold CV). A strong baseline is critical: Llama's low starting point leaves little room for SFT gains.

Model	Baseline	Fine-tuned	Δ	p
Llama 3.1 8B	37.5%	39.8%	+2.4	0.25
Qwen3-8B	**70.7%**	**73.0%**	**+2.3**	0.50

local gate additionally extracts key numbers from both versions and rejects the rephrase (with up to 2 retries) if any number is missing from the rephrased version.

3.4.2. Surface Diversity vs. Difficulty Scaling

We also experimented with generating harder questions using Claude Opus 4.6, requiring problems that combine multiple distributions, conditional probability, or system-level analysis. This produced 65 verified hard questions (cross-verified with GPT-5.4), forming dataset v2 (280 items).

The contrast in downstream performance is striking. Adding 65 hard questions to the 215-item base yielded only $+2.0\%$ improvement (Exp. 16), whereas adding 221 paraphrases of existing questions yielded $+6.2\%$ (Exp. 18), and scaling to 586 paraphrases yielded $+18.6\%$ (Exp. 24). This confirms the MetaMath finding: for small datasets, surface-level diversity in question formulation is more valuable than increasing problem difficulty.

3.5. Dataset Summary

Table 2 summarizes the dataset evolution. All versions contain numeric-only questions with chain-of-thought reasoning and verified numerical answers.

Each entry follows a chat-format structure with three roles: a system prompt establishing the reliability engineering expert persona, a user message containing the question, and an assistant message containing the chain-of-thought reasoning followed by a clearly delimited final answer.

4. Supervised Fine-Tuning

4.1. Base Model Selection

We evaluated two open-source instruction-tuned models as base candidates: Llama 3.1 8B [14] and Qwen3-8B [11], both in 4-bit quantized form via Unsloth. Table 3 shows that Qwen3-8B achieves a substantially higher zero-shot baseline on our reliability engineering questions (70.7% vs. 37.5%), likely due to its stronger mathematical pre-training. Moreover, fine-tuning Llama 3.1 8B caused a regression on numeric questions (-1.4pp) while improving only format-dependent categories (formula $+28.6$pp, text $+15.8$pp), suggesting catastrophic forgetting of mathematical capabilities. We therefore selected Qwen3-8B as our base model for all subsequent experiments.

By analyzing the model's thinking traces, we observed that the base model uses these blocks as *scratch work*: it explores solution avenues, backtracks when an approach seems wrong, and iterates toward the answer, much like a human solving a problem for the first time. Our SFT training data, by contrast, consists of polished step-by-step solutions that proceed directly to the correct answer without false starts. Training on these clean reasoning chains with thinking enabled would effectively require the model to "already know the answer" and go straight to a proof, conflicting with the exploratory nature of the thinking trace. We found that this mismatch caused the model to overfit, producing empty thinking tags followed by shallow pattern-matched answers. Disabling thinking during both training and inference resolved this: the model generates direct chain-of-thought responses consistent with the training format, while preserving its internal reasoning capabilities for use during GRPO (Section 5).

4.2. Training Setup

4.2.1. LoRA Configuration

We use 4-bit QLoRA [3] via the Unsloth framework, which provides optimized CUDA kernels for approximately 40% faster training. LoRA adapters [6] are applied to all attention and MLP projection layers. Key hyperparameters are rank $r = 16$, scaling factor $\alpha = 32$, and dropout 0.05.

4.2.2. Training Hyperparameters

Table 4 summarizes the training configuration. NEFTune [7] adds uniform noise to embedding vectors during training, serving as a regularizer against memorization on small datasets. We found $\alpha = 5$ to be optimal: $\alpha = 7$ degraded performance by -1.3pp (Exp. 17), and $\alpha = 10$ showed only marginal gains (Exp. 4). Early stopping with patience 2 (monitoring validation loss on a 10% held-out split) consistently converged to epoch 4 regardless of the maximum epoch setting (6 or 8), indicating a stable training dynamic.

4.2.3. Data Format

Training examples follow a three-turn chat format: a *system* message establishing the reliability engineering expert persona ("You are an expert in reliability engineering... Provide clear, accurate answers with step-by-step reasoning. At the end, clearly state your final answer after

Table 4. SFT training configuration (best setup).

Parameter	Value
Learning rate	2×10^{-4}
LR scheduler	Cosine
Optimizer	AdamW 8-bit
Batch size	2 (grad. accum. 4, eff. 8)
Max epochs	8 (early stop $\rightarrow$ epoch 4)
Warmup steps	5
Weight decay	0.01
NEFTune α	5
Precision	BF16
Max sequence length	2048 tokens

'Final Answer:' "), a *user* message containing the question, and an *assistant* message containing the full chain-of-thought reasoning followed by the final answer. The tokenizer's `apply_chat_template()` handles model-specific formatting.

4.3. Evaluation Protocol

4.3.1. K-Fold Cross-Validation

Given the small dataset sizes (215–866 examples), we use k-fold cross-validation rather than a single train/test split to obtain reliable performance estimates. Initial experiments used 5-fold CV (172 train / 43 test per fold), but 5 folds limit the minimum achievable p-value to 0.0625 under the Wilcoxon test, precluding statistical significance at $p < 0.05$. We therefore moved to 10-fold CV for experiments on 866 samples ($\sim$780 train / $\sim$87 test per fold), which achieves $p = 0.002$ when all folds show improvement.

For each fold, a fresh base model is loaded and fine-tuned from scratch to avoid information leakage between folds. All evaluation uses greedy decoding (`do_sample=False`) with `max_new_tokens=4096`, ensuring deterministic and reproducible results. Early experiments with stochastic decoding inflated improvements (e.g., $+7.0\%$ stochastic vs. $+2.3\%$ greedy in Exp. 3 vs. Exp. 12).

4.3.2. Answer Comparison

We extract the final answer from model responses using pattern matching ("Final Answer:", "The answer is:", "Therefore,") with fallback to the last non-empty line. Extracted answers are then compared against ground truth through a multi-level pipeline:

1. **Exact match:** Normalized strings (lowercase, strip whitespace, remove LaTeX wrappers such as `\boxed{}`, convert `\frac{a}{b}` to `a/b`).
2. **Numerical match:** Extract all numbers via regex; accept if all values match within 5% relative tolerance.

Table 5. SFT results across dataset sizes. Best configuration per dataset shown. All use Qwen3-8B, greedy decoding, LR=2×10^{-4}, NEFTune=5.

N	Folds	Ep.	Base	FT	Δ	p
215	5	4	70.7	73.0	$+2.3$	0.50
280	5	4	71.0	73.0	$+2.0$	0.69
501	5	4	59.1	65.3	$+6.2$	0.063
866	**10**	**8→4**	**63.6**	**82.2**	$\mathbf{+18.6}$	**0.002**

3. **Fraction-to-decimal:** Convert fractions (e.g., 14/33 $\leftrightarrow$ 0.4242).
4. **Percentage conversion:** Normalize percentages (e.g., 25.5% $\leftrightarrow$ 0.255).
5. **Partial match:** Substring containment for answers longer than 3 characters.

An answer is considered correct if any of the above criteria is satisfied.

4.3.3. Statistical Testing

We use the Wilcoxon signed-rank test to compare per-fold baseline and fine-tuned accuracies. This non-parametric paired test is appropriate because: (1) we have a small number of paired observations (5 or 10 folds), (2) we cannot assume normality of accuracy differences, and (3) the test is robust to outlier folds. We use $p < 0.05$ as the significance threshold.

We additionally track *question flips* wherein a question changes correctness between baseline and fine-tuned model, aggregated across all folds. A successful fine-tuning should show substantially more wrong$\rightarrow$right flips than right$\rightarrow$wrong regressions.

4.4. SFT Results

Table 5 presents the main SFT results across dataset sizes. The improvement scales consistently with dataset size: from $+2.3\%$ (not significant) on 215 original questions to $+18.6\%$ ($p = 0.002$) on 866 augmented questions. All 10 folds showed improvement in Exp. 24, with per-fold deltas ranging from $+12.6\%$ to $+26.4\%$.

The question flip analysis for Exp. 24 shows 206 wrong$\rightarrow$right flips against only 45 right$\rightarrow$wrong regressions (ratio 4.6:1), indicating that fine-tuning broadly improves performance rather than trading one type of error for another. By contrast, Exp. 12 (215 samples) showed only 23 wrong$\rightarrow$right vs. 18 right$\rightarrow$wrong (ratio 1.3:1), confirming that larger augmented datasets produce more decisive improvements.

Note that the baseline accuracy is lower on larger datasets (63.6% on 866 vs. 70.7% on 215) because the augmented test sets include paraphrased questions the base model has not seen. This makes the improvement more im-

Table 6. Ablation studies on 215 numeric questions (5-fold CV, greedy). Bold marks the best configuration.

Variation	Δ	p
Best (LR=2×10^{-4}, NEFTune=5, r=16, 4ep)	**+2.3**	0.50
Lower LR (1×10^{-4})	−0.5	1.0
Higher NEFTune ($\alpha = 7$)	−1.3	0.38
Lower LoRA rank ($r = 8$, $\alpha = 16$)	+1.4	0.88
3 epochs (vs. 4)	+0.9	0.88
5 epochs (vs. 4)	+1.9	0.50
DPO (instead of SFT)	−1.7	0.63
SFT + GRPO	+0.5	0.88

pressive: the fine-tuned model handles both original and paraphrased formulations effectively.

4.5. Ablation Studies

Table 6 summarizes key ablation results on the 215-question dataset (5-fold CV, greedy decoding).

Early stopping. Across all experiments with maximum epochs $\geq$ 6, early stopping (patience 2, monitoring validation loss) consistently selects epoch 4 as the best checkpoint. This holds for both 501-sample (Exp. 19) and 866-sample (Exp. 22, 24) experiments, suggesting a stable convergence point for this model and domain.

Learning rate. LR=2×10^{-4} consistently outperforms LR=1×10^{-4}, consistent with the recommendation that LoRA adapters benefit from learning rates approximately $10\times$ higher than full fine-tuning.

RL methods on small data. Both DPO (-1.7%) and SFT+GRPO ($+0.5\%$) underperformed pure SFT ($+2.3\%$) on 215 questions. DPO actually degraded performance, likely because the small dataset produced too few informative preference pairs. GRPO showed marginal gains but insufficient to justify the additional training complexity at this scale.

Note that the SFT+GRPO result ($+0.5\%$) reported here used an early version of GRPO on 215 questions with TRL and LLM-judge rewards. Our improved GRPO pipeline (Section 5), using a custom training loop with dynamic sampling, verifiable numerical rewards, and a larger 266-question mixed-difficulty training set, achieves substantially better results (+7.5pp over base).

5. Reinforcement Learning with GRPO

5.1. Motivation

While SFT teaches the model domain-specific formatting and reasoning patterns, it is fundamentally limited to imitating the training data. On questions the base model already solves correctly, SFT reinforces existing capabilities; on questions where the model fails, SFT provides the correct answer but does not teach the model *how to arrive at it* from its own reasoning process. This limitation is reflected in our results: the SFT model trained on the 600 "easy" questions (those the base model solves 4/4 times) shows no improvement and even slight regression on the remaining 266 harder questions (Section 5.7).

Group Relative Policy Optimization (GRPO) [15] offers a complementary approach: rather than imitating expert solutions, the model generates multiple candidate answers and learns to prefer reasoning chains that lead to correct results. For reliability engineering, where answers are verifiable numerical values, this provides a natural reward signal without requiring a learned reward model.

5.2. GRPO Algorithm

GRPO generates G completions $\{o_1, \ldots, o_G\}$ for each prompt q, scores each with a reward function $r(o_i, q)$, and computes *group-relative* advantages:

$$\hat{A}_i = \frac{r(o_i, q) - \bar{r}}{\sigma_r + \epsilon}, \tag{1}$$

where $\bar{r}$ and σ_r are the mean and standard deviation of rewards within the group. The policy is updated to maximize:

$$\mathcal{L}_{\text{GRPO}} = -\frac{1}{G}\sum_{i=1}^{G} \hat{A}_i \cdot \frac{\log \pi_\theta(o_i \mid q)}{|o_i|}, \tag{2}$$

where $|o_i|$ is the response length in tokens, normalizing the loss to prevent bias toward shorter completions (Dr. GRPO [2]).

Unlike PPO, GRPO requires no critic model: the group mean serves as the baseline. Unlike DPO, it does not require pre-computed preference pairs but generates them online. This makes GRPO particularly suited to domains with verifiable rewards and limited data.

5.3. Dynamic Sampling (DAPO)

A key challenge with GRPO on small datasets is that many prompts produce zero reward variance: either all G generations are correct (no learning signal for improvement) or all are wrong (no positive signal). Following DAPO (Dynamic sampling Policy Optimization) [25], we implement *dynamic sampling*: when all G generations produce the same reward, the prompt is discarded and a new one is sampled. This ensures every training step provides a meaningful gradient signal.

We track *useful steps* (prompts with reward variance) separately from *wasted steps* (prompts discarded due to zero variance). The waste rate serves as a diagnostic: a rising waste rate indicates the model is converging (more prompts

become too easy or too hard), while an initially high waste rate suggests the training distribution is poorly calibrated to the model's current capability.

5.4. Reward Design

Our reward function uses the same automated numerical comparison as the SFT evaluation pipeline (Section 4.3). Given a model response o and ground-truth target y, we extract the last `\boxed{}` value $\hat{y}$ and compute a graded reward based on relative error $\epsilon = |\hat{y} - y|/(|y| + 10^{-9})$:

$$r(o, q) = \begin{cases} 1.0 & \text{if } \epsilon \leq 0.1\% \\ 0.8 & \text{if } \epsilon \leq 1\% \\ 0.4 & \text{if } \epsilon \leq 5\% \\ 0.01 & \text{if } \epsilon > 5\% \\ 0.0 & \text{if no } \texttt{\textbackslash boxed\{\}} \text{ found} \\ -0.3 & \text{if } > 2 \text{ boxed values} \end{cases} \tag{3}$$

The graded structure provides a smoother reward landscape than binary correct/incorrect: partially correct answers ($\epsilon \leq 5\%$) receive positive reward, encouraging the model to refine its computation rather than abandon an approach entirely. The penalty for multiple `\boxed{}` values discourages hedging.

5.5. Training Data Split

The 866-question dataset (v4) is split based on base-model screening: each question is evaluated 4 times with the base Qwen3-8B model, and questions are categorized by success rate:

- **600 questions (4/4 correct):** Used for SFT training. These are "easy" for the base model; SFT reinforces correct patterns.
- **266 questions (1/4 to 3/4 correct):** Used for GRPO training. These have natural reward variance—some generations succeed, others fail—making them ideal for group-relative optimization.

Questions where the base model scores 0/4 are excluded from GRPO as they provide no positive reward signal. This split ensures zero data leakage between SFT and GRPO stages and targets GRPO training at the model's learning frontier—questions that are neither trivially easy nor impossibly hard.

5.6. Training Configuration

Table 7 summarizes the GRPO training configuration. We use a custom training loop (no TRL dependency) with memory-efficient per-generation backward passes and Unsloth's optimized inference for the generation phase.

The pipeline proceeds as: (1) load the base Qwen3-8B model, (2) load the SFT LoRA adapter from fold_4, (3) enable gradients only on LoRA parameters (~43.6M trainable), (4) run the GRPO loop with dynamic sampling on the 266 mixed questions. Thinking mode is disabled (`/no_think` system prompt) to maintain consistency with the SFT training format.

Table 7. GRPO training configuration (baseline, lr=1×10^{-5}).

Parameter	Value
Base model	Qwen3-8B (4-bit)
SFT initialization	SFT fold_4 LoRA ($r = 16$, $\alpha = 32$)
Generations per prompt (G)	4
Max new tokens	2048
Useful steps	200
Max total attempts	600
Max resample attempts/step	10
Learning rate	1×10^{-5}
Gradient accumulation	4
Max gradient norm	0.1
KL penalty (β)	0.0
Checkpoint interval	every 20 useful steps
Optimizer	AdamW (weight decay 0.1)
LR scheduler	Linear warmup + cosine decay
Warmup steps	15

Table 8. Accuracy on 266 mixed questions (GRPO training set). Δ is vs. Qwen3-8B base.

Model	Correct	Acc.	Trunc.	Δ
Qwen2.5-7B base	~40	15.0%	0	—
Qwen3-8B base	134	50.4%	4	—
SFT fold_0	124	46.6%	7	−3.8
SFT fold_4	135	50.8%	8	+0.4
GRPO from scratch, step 80	135	50.8%	1	+0.4
GRPO from SFT, step 200 (lr=1e-5)	139	52.3%	6	+1.9
GRPO from SFT, step 100 (lr=1e-5)	144	54.1%	9	+3.7
GRPO from SFT, step 80 (lr=1e-5)	**154**	**57.9%**	10	**+7.5**

5.7. GRPO Results

5.7.1. In-Distribution Evaluation (266 Mixed Questions)

Table 8 presents accuracy on the 266 GRPO training questions across all model configurations. All evaluations use deterministic seeding (seed $= 3407 + q_i$) with temperature 1.0 and top-p 0.95, matching the GRPO generation settings.

The best configuration (GRPO from SFT at checkpoint 80) achieves **57.9%** accuracy, a +7.5pp improvement over the Qwen3-8B base and +7.1pp over the SFT fold_4 starting point. Performance degrades with continued training: −3.8pp from step 80 to step 100 and −5.6pp to step 200, indicating overfitting after approximately 80 useful steps on this 266-question dataset.

Table 9. Accuracy on 54 independent holdout questions. These questions were never seen during SFT or GRPO training.

Model	Correct	Acc.	Trunc.	Δ
Qwen3-8B base	29	53.7%	1	—
SFT fold_4	18	33.3%	2	−20.4
GRPO from SFT, step 80	29	53.7%	2	+0.0

Table 10. Ablation: effect of SFT warm-start on GRPO (step 80, 266 questions).

Configuration	Acc.	Δ vs. Base	Waste Rate
GRPO from scratch (fresh LoRA)	50.8%	+0.4	29.3%
GRPO from SFT (fold_4 LoRA)	**57.9%**	**+7.5**	23.1%

5.7.2. Out-of-Distribution Evaluation (54-Question Holdout)

Table 9 evaluates generalization on 54 independent holdout questions never seen during SFT or GRPO training.

The holdout results show that **GRPO preserves base-model generalization**: GRPO from SFT at step 80 achieves 53.7%, matching the base model exactly. While the +7.5pp in-distribution gain does not transfer to holdout, crucially it does not regress either. More importantly, GRPO fully recovers the catastrophic forgetting caused by SFT: the SFT fold_4 alone drops to 33.3% (−20.4pp) on the holdout, while the GRPO-trained model recovers to 53.7%.

Caveat on holdout size. With only 54 questions, each question represents ∼1.85pp of accuracy, limiting the statistical power of these conclusions. We deliberately chose not to augment the holdout via paraphrasing: while this would increase sample size, it would introduce repeated mathematical content and risk inflating accuracy through surface-level memorization rather than genuine reasoning. A larger holdout composed of *distinct* problems would strengthen these findings.

5.7.3. Ablation: SFT Warm-Start

To isolate the contribution of SFT initialization, we run an identical GRPO configuration but starting from a *fresh* (zero-initialized) LoRA instead of the SFT checkpoint.

The SFT warm-start is essential: GRPO from scratch barely exceeds the base model (+0.4pp, likely within noise), while GRPO from the SFT checkpoint gains +7.5pp. The SFT initialization also produces a lower waste rate (23.1% vs. 29.3%), indicating that the SFT-adapted model produces more diverse reward signals, making dynamic sampling more efficient. This is paradoxical: the SFT alone degrades holdout performance by −20.4pp, yet it provides a critical initialization for GRPO.

Table 11. Learning rate comparison (all from SFT fold_4, 200 useful steps).

Configuration	LR	Waste Rate	Total Attempts	Best
GRPO conservative LR	5×10^{-6}	21.3%	254	Eval
GRPO baseline LR	$\mathbf{1 \times 10^{-5}}$	23.1%	260	**57.9%**
GRPO aggressive LR	2×10^{-5}	20.9%	253	Eval

We hypothesize that SFT teaches the model the *format and vocabulary* of domain-specific reasoning (e.g., proper use of reliability formulas, structured step-by-step computation) without teaching correct *application* to hard problems. GRPO then leverages this structural foundation to optimize for numerical correctness. Without SFT, the base model's LoRA weights start at zero, and GRPO must simultaneously learn both format and reasoning—too many degrees of freedom for the small 266-question training set.

5.7.4. Learning Rate Ablation

Training dynamics are similar across learning rates, with waste rates between 20.9% and 23.1%. Full evaluation results for the conservative and aggressive learning rate configurations will determine whether the learning rate significantly affects peak accuracy or the onset of overfitting.

6. Experiments and Analysis

6.1. Comparison with Frontier Models

To contextualize our fine-tuned 8B model's performance, we evaluate three frontier models on two benchmarks: (1) a 500-question sample from our training dataset (v4), and (2) a fully independent 54-question holdout set composed of textbook problems, professor-authored exam questions, and French reliability exercises (*TD fiabilité*), none of which appear in the training data. For all models, we use the same answer extraction and comparison pipeline described in Section 4.3. When the extraction pipeline fails to parse a model's response (UNABLE_TO_EXTRACT), we exclude that question from the denominator rather than counting it as incorrect, to avoid penalizing models for formatting differences.

On the v4 sample (Table 12), Claude Sonnet 4.6 leads at 92.4%, followed by o3-mini (86.4%) and Gemini 3.1 Pro (76.4%). Our fine-tuned Qwen3-8B achieves 82.2% under cross-validation on the same dataset, placing it between o3-mini and Gemini 3.1 Pro despite being an order of magnitude smaller. Note that the fine-tuned model's accuracy is measured via 10-fold CV (training on 90%, testing on 10%) and is therefore not directly comparable to the frontier models which see all questions at test time; the comparison is nevertheless informative as an approximate ranking.

On the independent holdout (Table 13), Claude Sonnet 4.6 achieves 81.1% and o3-mini 77.1%. Gemini 3.1

Table 12. Frontier model accuracy on our reliability engineering benchmarks. N_{eval} is the number of questions with successfully extracted answers.

Model	v4 500-sample		
	Correct	N_{eval}	Accuracy
Claude Sonnet 4.6	462	500	**92.4%**
o3-mini	427	494	86.4%
Gemini 3.1 Pro Preview	308	403	76.4%
Qwen3-8B (base)	63.6% (10-fold CV)		
Qwen3-8B (SFT only)	82.2% (10-fold CV)		
Qwen3-8B (SFT+GRPO)	**57.9%** on 266 hard q.		

Table 13. Independent holdout evaluation (54 questions, no overlap with training data).

Model	Holdout (54 questions)		
	Correct	N_{eval}	Accuracy
Claude Sonnet 4.6	43	53	**81.1%**
o3-mini	37	48	77.1%
Gemini 3.1 Pro Preview	19	23	82.6%
Qwen3-8B (base)	29	54	53.7%
Qwen3-8B (SFT fold_4)	18	54	33.3%
Qwen3-8B (SFT+GRPO)	29	54	53.7%

Pro achieves 82.6% on the 23 questions where answer extraction succeeded, though the high extraction failure rate (31/54 = 57.4%) limits the reliability of this estimate. Our base Qwen3-8B achieves 53.7%, but the SFT model regresses sharply to 33.3% (-20.4pp), revealing severe catastrophic forgetting on truly unseen questions. GRPO fully recovers to 53.7%, matching the base model and demonstrating that RL training restores generalization lost during SFT while adding in-distribution gains (Section 5.7).

6.2. General Reasoning Evaluation

A common concern with domain-specific fine-tuning is catastrophic forgetting of general capabilities. To assess whether our SFT and GRPO pipeline preserves or improves general reasoning, we evaluate the base and fine-tuned models on standard mathematical reasoning benchmarks.

Our holdout evaluation (Table 13) provides direct evidence of catastrophic forgetting: SFT fold_4 drops from 53.7% to 33.3% on 54 unseen reliability engineering questions. This is consistent with the observation that SFT was trained exclusively on 600 "easy" questions (where the base model already scores 4/4), which teaches surface patterns without improving hard-problem reasoning. GRPO partially recovers this regression (50.0%), suggesting that RL training restores some of the base model's general reasoning capability lost during SFT.

Table 14. SFT improvement and question flips across dataset sizes. W→R: wrong→right, R→W: right→wrong.

N	Δ	p	W→R	R→W	Ratio
215	+2.3	0.50	23	18	1.3
280	+2.0	0.69	19	15	1.3
501	+6.2	0.063	74	43	1.7
866	**+18.6**	**0.002**	**206**	**45**	**4.6**

Table 15. Effect of training epochs on 866-question dataset (10-fold CV).

Epochs	FT Accuracy	Δ	p
3	75.1%	+11.4	0.002
4	79.3%	+15.7	0.002
6 (ES→4)	81.8%	+18.1	0.002
8 (ES→4)	82.2%	+18.6	0.002

6.3. Analysis

6.3.1. Dataset Size Scaling

Figure **??** plots SFT improvement as a function of training set size. The relationship is clearly non-linear: improvements are marginal below 300 examples (+2.0–2.3%), begin to emerge at 501 examples (+6.2%, $p = 0.063$), and become large and statistically significant at 866 examples (+18.6%, $p = 0.002$). This suggests a threshold effect around 500 examples, below which LoRA fine-tuning on this domain struggles to overcome the regularization imposed by the frozen base weights.

[TODO: Add scaling curve figure.]

The question flip ratio (Table 14) is particularly informative: at 215 samples, the model trades nearly as many correct answers as it gains (1.3:1), while at 866 samples, gains overwhelm regressions (4.6:1). This indicates that larger augmented datasets not only improve average performance but also make the improvement more *robust*, with fewer catastrophic regressions on individual questions.

6.3.2. Epoch Convergence

Table 15 shows SFT improvement as a function of training epochs on the 866-question dataset (10-fold CV). Performance increases rapidly from 3 to 4 epochs, then plateaus. Early stopping with patience 2 independently selects epoch 4 in both Exp. 22 (max 6) and Exp. 24 (max 8), confirming this as a stable convergence point.

6.3.3. Error Analysis

A detailed error analysis by sub-topic is left for future work. We note that the GRPO model's truncation rate increases with training (4 for base, 10 for ckpt-80), suggesting that GRPO encourages longer reasoning chains that occasionally exceed the generation limit. The holdout ac-

curacy matches the base model exactly (53.7%), suggesting that GRPO improves in-distribution performance without degrading general reasoning—unlike SFT alone, which causes severe holdout regression.

7. Conclusion

We presented a complete pipeline for adapting a general-purpose LLM to reliability engineering, a technical domain with no existing training datasets or benchmarks. Starting from 56 textbook-extracted seed questions, our adapted self-instruct methodology with cross-model answer verification and paraphrase augmentation scales the dataset to 866 verified examples. Supervised fine-tuning of Qwen3-8B yields a statistically significant $+18.6\%$ accuracy improvement ($p = 0.002$) under 10-fold cross-validation, placing our 8B-parameter model between frontier models o3-mini and Gemini 3.1 Pro on domain-specific tasks.

Our analysis reveals two key findings for the synthetic data generation community. First, the *self-instruct ceiling effect*: LLM generators create questions within their own capability envelope, making same-model answer consistency a trivially satisfied check rather than a meaningful quality filter. Second, paraphrase augmentation—creating surface-level rephrases while preserving mathematical content—is far more effective than generating harder synthetic questions, consistent with MetaMath [24] and suggesting that diversity of expression matters more than difficulty in small-data regimes.

GRPO with verifiable rewards achieves an additional $+7.5$pp on the 266 mixed-difficulty training questions, demonstrating that reinforcement learning can improve reasoning beyond SFT on verifiable tasks. Our ablation study reveals a paradoxical role for the SFT stage: while SFT alone causes catastrophic forgetting on unseen questions (-20.4pp on holdout), it is essential as initialization for GRPO ($+7.1$pp vs. GRPO from scratch). We hypothesize that SFT provides structural alignment (domain vocabulary, format, formula usage) that GRPO requires as a foundation for reasoning optimization.

Importantly, GRPO preserves base-model generalization: on the 54-question independent holdout, the GRPO model matches the base model (53.7%), fully recovering the severe catastrophic forgetting caused by SFT alone (-20.4pp). This suggests that GRPO provides in-distribution gains without the out-of-distribution cost typically associated with fine-tuning on small datasets.

Limitations. Our evaluation is limited to numeric questions with automated answer comparison; formula derivation and conceptual questions require human evaluation. The 54-question holdout is small (each question ≈ 1.85pp), limiting statistical power for out-of-distribution conclusions. The self-instruct ceiling effect means our synthetic data may not cover the hardest problems in reliability engineering.

Future work. Promising directions include: (1) scaling the GRPO training set via paraphrase augmentation of the 266 mixed questions, following the successful SFT scaling pattern; (2) introducing a KL penalty ($\beta > 0$) to reduce overfitting and preserve base model generalization; (3) early stopping based on holdout validation rather than training loss; (4) grounding data generation in textbook source material (following SearchInstruct [8]); and (5) extending to non-numeric question types and other engineering domains.

References

[1] Jiaxi Cui, Zongjian Li, Yang Yan, Bohua Chen, and Li Yuan. ChatLaw: Open-source legal large language model with integrated external knowledge bases. *arXiv preprint arXiv:2306.16092*, 2023. 2

[2] DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025. 3, 8

[3] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized language models. *NeurIPS*, 2023. 3, 6

[4] Bosheng Ding et al. Data augmentation using large language models: Data perspectives, learning paradigms and challenges. *arXiv preprint arXiv:2403.02990*, 2024. 3

[5] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. *NeurIPS*, 2021. 1

[6] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *ICLR*, 2022. 3, 6

[7] Neel Jain, Ping-yeh Chiang, Yuxin Wen, John Kirchenbauer, Hong-Min Chu, Gowthami Somepalli, Brian R. Bartoldson, Bhavya Kailkhura, Avi Schwarzschild, Aniruddha Saha, et al. NEFTune: Noisy embeddings improve instruction finetuning. *arXiv preprint arXiv:2310.05914*, 2023. 3, 6

[8] Xiao Li et al. SearchInstruct: Domain-specific instruction data generation via RAG. *arXiv preprint arXiv:2509.10708*, 2025. 2, 12

[9] OpenAI. GPT-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023. 1

[10] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *NeurIPS*, 2022. 3

[11] Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025. 6

[12] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct

preference optimization: Your language model is secretly a reward model. *NeurIPS*, 2023. 3

[13] Leonardo Ranaldi and André Freitas. Self-refine instruction-tuning for aligning reasoning in language models. 2024. 2

[14] Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, et al. Code Llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*, 2024. 2, 6

[15] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Mingchuan Zhang, Y.K. Li, Y. Wu, and Daya Guo. DeepSeek-Math: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024. 3, 8

[16] Ilia Shumailov, Zakhar Shumaylov, Yiren Zhao, Nicolas Papernot, Ross Anderson, and Yarin Gal. The curse of recursion: Training on generated data makes models forget. *Nature*, 631:755–759, 2024. 3

[17] Karan Singhal, Tao Tu, Juraj Gottweis, Rory Sayres, Ellery Wulczyn, Le Hou, Kevin Clark, Stephen Pfohl, Heather Cole-Lewis, Darlene Neal, et al. Towards expert-level medical question answering with large language models. *arXiv preprint arXiv:2305.09617*, 2023. 1, 2

[18] Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. Stanford Alpaca: An instruction-following LLaMA model. *GitHub repository*, 2023. 2

[19] Jing Wang et al. PersonaMath: Enhancing math reasoning through persona-driven data augmentation. *arXiv preprint arXiv:2410.01504*, 2024. 3, 5

[20] Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A. Smith, Daniel Khashabi, and Hannaneh Hajishirzi. Self-instruct: Aligning language models with self-generated instructions. In *ACL*, 2023. 2, 4

[21] Chaoyi Wu et al. PMC-LLaMA: Toward building open-source language models for medicine. *Journal of the American Medical Informatics Association*, 2024. 2

[22] Can Xu, Qingfeng Sun, Kai Zheng, Xiubo Geng, Pu Zhao, Jiazhan Feng, Chongyang Tao, and Daxin Jiang. WizardLM: Empowering large language models to follow complex instructions. *arXiv preprint arXiv:2304.12244*, 2023. 2

[23] Haoran Yu et al. CoT-self-instruct: Chain-of-thought self-instruct for aligning language models. *arXiv preprint arXiv:2405.00402*, 2024. 2, 4, 5

[24] Longhui Yu, Weisen Jiang, Han Shi, Jincheng Yu, Zhengying Liu, Yu Zhang, James T. Kwok, Zhenguo Li, Adrian Weller, and Weiyang Liu. MetaMath: Bootstrap your own mathematical questions for large language models. *arXiv preprint arXiv:2309.12284*, 2024. 1, 3, 5, 12

[25] Qiying Yu, Zheng Sun, Huawen Hou, Kaituo Yu, et al. DAPO: An open-source LLM reinforcement learning system via dynamic sampling. *arXiv preprint arXiv:2503.14476*, 2025. 2, 3, 8

A. Complete Experiment Log

Table 16 lists all 26 experiments with full configurations and results. All experiments use 4-bit QLoRA via Unsloth on NVIDIA A100-SXM4-40GB GPUs. Evaluation uses greedy decoding (`do_sample=False`) from Exp. 7 onwards; earlier experiments used stochastic sampling (marked with †). Bold rows indicate best results per dataset size. ES = early stopping (patience=2); the notation "8(ES→4)" means max 8 epochs with early stopping selecting epoch 4.

Table 17 lists all GRPO experiments. All use the custom DAPO-style training loop on 266 mixed-difficulty questions (unless noted), with Qwen3-8B 4-bit and LoRA $r = 16/\alpha = 32$. Evaluation uses deterministic seeding (seed $= 3407 + q_i$), temperature 1.0, top-p 0.95.

Notes: Exp. 1 uses Llama 3.1 8B; all others use Qwen3-8B. Exp. 5 uses LoRA $r = 8/\alpha = 16$; all others use $r = 16/\alpha = 32$. Exp. 10 timed out after 14 hours. Exp. 13 crashed due to Triton kernel incompatibility. Exp. 14 completed only 4 of 5 folds (missing adapter for fold 4). Accuracy values are percentages; Δ is the absolute difference (FT − Baseline). W→R and R→W are question flip counts aggregated across all folds.

B. Key Prompts

Below are selected prompts used in our data pipeline. Minor formatting adjustments have been made for readability.

B.1. Synthetic Question Generation

Used to generate new reliability engineering problems from seed examples (Section 3.2).

> *You are an expert in reliability engineering, creating practice problems for students.*
>
> *I will show you {num_examples} example problems. Your task is to:*
>
> 1. *Understand the style, difficulty, and topics of these examples*
> 2. *Create {num_to_generate} new, original problems that are similar in style and difficulty*
> 3. *Make questions that are educational and challenging*
> 4. *Provide complete solutions with step-by-step reasoning*
>
> *Important: Create different questions (don't just change numbers in the examples). Use similar concepts but new scenarios. Include all necessary data in the question (self-contained). Show clear reasoning steps. Provide specific numerical answers when appropriate.*
>
> *[3 randomly sampled seed examples shown]*
>
> *For each problem, provide:*
> *QUESTION: [Complete problem statement]*
> *REASONING: [Step-by-step solution]*
> *ANSWER: [Final answer(s)]*

B.2. Paraphrase Augmentation

Used to rephrase existing questions while preserving mathematical content (Section 3.4). The prompt includes two manually curated few-shot examples (omitted for brevity).

> *You are a technical writer specializing in reliability engineering. Your task is to rephrase a question and its solution reasoning while preserving their exact mathematical meaning.*
>
> *Strict rules:*
>
> 1. *All numerical values, failure rates, probabilities, time periods, and parameters must remain exactly the same*
> 2. *All formulas, equations, and mathematical relationships must be preserved*
> 3. *The correct answer must be identical*
> 4. *Change the wording, sentence structure, and phrasing to create a distinct version*
> 5. *Do not add information, assumptions, or conditions not present in the original*
>
> *What to change: Synonyms for non-technical words (e.g., "compute" → "determine"), sentence structure, scenario framing (e.g., "plant" → "factory"), transition words.*
>
> *What must not change: Any number, rate, probability, or parameter value. Variable names in formulas. The mathematical operations and formulas used. The final answer.*
>
> *[2 few-shot examples shown]*
>
> *Respond with a JSON object:* `{"question": "...", "reasoning": "..."}`

B.3. Paraphrase Verification

Used by GPT-5.4 to verify that a rephrase preserves mathematical meaning (Section 3.4).

> *You are verifying whether a rephrased reliability engineering question preserves the exact mathematical meaning of the original. Compare the original and rephrased versions. Check all of the following:*
>
> 1. *Numerical values: Are all numbers, rates, probabilities exactly the same?*
> 2. *Mathematical meaning: Do both questions ask for the exact same quantity?*
> 3. *Constraints: Are all conditions and given information preserved?*
> 4. *Solution: Would the exact same calculation steps produce the correct answer?*
> 5. *Reasoning: Are all formulas and intermediate values identical?*

Table 16. Complete experiment log. All models are Qwen3-8B (4-bit) except Exp. 1 (Llama 3.1 8B). LoRA config is r/α with dropout 0.05 unless noted. † = stochastic eval (inflated). ‡ = thinking enabled (mismatch).

Exp	Dataset	N	Folds	Ep.	LR	NEF	Method	Baseline	FT	Δ	W→R	R→W	p
Round 1: Model Selection													
1	cleaned (all)	256	5	3	2e-4	–	SFT†	37.5	39.8	+2.4	35	29	0.25
2	cleaned (num)	215	5	4	5e-5	–	SFT†‡	67.9	67.0	−0.9	25	27	0.88
Round 2: Hyperparameter Search (stochastic eval)													
3	cleaned (num)	215	5	3	2e-4	5	SFT†	67.9	74.9	+7.0†	31	16	0.13
4	cleaned (num)	215	5	3	2e-4	10	SFT†	70.7	72.1	+1.4	21	18	1.0
5	cleaned (num)	215	5	3	2e-4	5	SFT†	67.9	69.3	+1.4	24	21	0.88
6	cleaned (num)	215	5	3	1e-4	5	SFT†	68.8	68.4	−0.5	20	21	1.0
Round 3: Deterministic Evaluation (greedy decoding)													
7	v2 (num)	280	5	3	2e-4	5	SFT	64.0	65.2	+1.3	21	21	0.88
8	v2 (num)	280	5	5	2e-4	5	SFT	64.3	65.7	+1.4	30	19	0.88
9	cleaned (num)	215	5	5	2e-4	5	SFT†	70.7	72.6	+1.9	21	17	0.50
11	cleaned (num)	215	5	3	2e-4	5	SFT	70.7	71.6	+0.9	22	20	0.88
12	**cleaned (num)**	**215**	**5**	**4**	**2e-4**	**5**	**SFT**	**70.7**	**73.0**	**+2.3**	**23**	**18**	**0.50**
16	**v2 (num)**	**280**	**5**	**4**	**2e-4**	**5**	**SFT**	**71.0**	**73.0**	**+2.0**	**19**	**15**	**0.69**
17	cleaned (num)	215	5	4	2e-4	7	SFT	71.0	69.8	−1.3	19	18	0.38
Round 4: Alternative Methods													
10	v2 (num)	280	5	DPO	5e-6	–	DPO	64.3	–	–	–	–	–
14	cleaned (num)	215	5	DPO	5e-6	–	DPO	69.8	68.0	−1.7	4	7	0.63
13	cleaned (num)	215	5	SFT+GRPO	2e-4	5	GRPO	70.7	–	–	–	–	–
15	cleaned (num)	215	5	SFT+GRPO	2e-4	5	GRPO	70.7	71.2	+0.5	18	17	0.88
Round 5: Paraphrase Augmentation (5-fold CV)													
18	**v3 (num)**	**501**	**5**	**4**	**2e-4**	**5**	**SFT**	**59.1**	**65.3**	**+6.2**	**74**	**43**	**0.063**
19	v3 (num)	501	5	8(ES→4)	1.5e-4	5	SFT	59.1	64.5	+5.4	83	56	0.063
20	v3 (num)	501	5	6	1e-4	5	SFT	59.1	64.9	+5.8	78	49	0.063
Round 6: Scaled Augmentation (10-fold CV)													
23	v4 (num)	866	10	3	2e-4	5	SFT	63.6	75.1	+11.4	171	72	**0.002**
21	v4 (num)	866	10	4	2e-4	5	SFT	63.6	79.3	+15.7	194	58	**0.002**
22	v4 (num)	866	10	6(ES→4)	2e-4	5	SFT	63.6	81.8	+18.1	208	51	**0.002**
24	**v4 (num)**	**866**	**10**	**8(ES→4)**	**2e-4**	**5**	**SFT**	**63.6**	**82.2**	**+18.6**	**206**	**45**	**0.002**
Round 7: 600-Sample Subset Validation (10-fold CV)													
25	**v4 sub (num)**	**600**	**10**	**5**	**2e-4**	**5**	**SFT**	**68.0**	**79.7**	**+11.7**	**108**	**38**	**0.002**
26	v4 sub (num)	600	10	5(ES)	2e-4	5	SFT	68.0	78.8	+10.8	100	35	**0.002**

Table 17. GRPO experiment log. All models are Qwen3-8B (4-bit) with 200 useful steps, $G = 4$ generations, grad_accum=4. Accuracy is on the 266 mixed questions unless noted.

Name	Init	LR	Waste%	Attempts	Step 80	Step 100	Step 200	Holdout (54q)	Notes
GRPO baseline	SFT fold_4	1e-5	23.1	260	**57.9%**	54.1%	52.3%	53.7% (step 80)	Best config
GRPO aggressive LR	SFT fold_4	2e-5	20.9	253	Evaluation pending			—	2× LR
GRPO conservative LR	SFT fold_4	5e-6	21.3	254	Evaluation pending			—	0.5× LR
GRPO from scratch	Fresh LoRA	1e-5	29.3	283	50.8%	Pending	Pending	Pending	No SFT init

Respond with a JSON object:

```
{"values_preserved": bool,
"meaning_preserved": bool,
"constraints_preserved": bool,
"solution_equivalent": bool,
"reasoning_equivalent": bool,
"overall_valid": bool, "issues": "..."}
```

B.4. GRPO System Prompt

Used during GRPO training and evaluation (Section 5). The `/no_think` prefix disables Qwen3's internal thinking mode for consistency with SFT format.

`/no_think`
You are a Reliability Engineering Expert. Solve the user's problem step-by-step with rigorous mathematical reasoning.

Rules for your final answer:

- *Write ONE single* `\boxed{}` *at the very end of your response — your final answer only.*
- *Do NOT use* `\boxed{}` *for intermediate steps or calculations.*

Always put your single final numerical answer inside `\boxed{}`.

B.5. SFT System Prompt

Used as the system message during both training and evaluation (Section 4.2).

You are an expert in reliability engineering, probability theory, and system analysis. Provide clear, accurate answers with step-by-step reasoning. At the end, clearly state your final answer after 'Final Answer:'.